

RSA Lesson 1: Quiz Preparation Notes

Extended Euclid (Table Method) · Euler's Theorem · Chinese Remainder Theorem (Methods A and C)
Construction and Correctness of RSA · Four Attacks on RSA

Section	What it covers
1	Overview: the problem RSA solves, and why the sections come in this order
2	Modular arithmetic and the Extended Euclidean Algorithm: gcd, Bézout, the table method, modular inverses
3	Euler's totient function, Euler's theorem, Fermat's little theorem
4	Chinese Remainder Theorem: the theorem, then solving systems by hand (Method A: substitution, Method C: two-congruence formula)
5	RSA: why decryption undoes encryption, key generation, correctness for every message, fast exponentiation, CRT decryption
6	Four attacks: Håstad, Common Modulus, Shared Primes, Fermat's factorization

Contents

1	Overview: The Problem RSA Solves, and the Order of the Tools	3
2	Modular Arithmetic and the Extended Euclidean Algorithm	4
2.1	Modular Arithmetic in 60 Seconds	4
2.2	The ordinary Euclidean algorithm	4
2.3	Bézout's identity: the gcd is a combination	4
2.4	The table method	5
2.5	From Bézout to the modular inverse	6
3	Euler's Totient Function, Euler's Theorem and Fermat's Little Theorem	9
3.1	Euler's Totient Function: Counting the "Well-Behaved" Numbers	9
3.2	Why We Care: The Coprime Numbers Form a Self-Contained "Club"	10
3.3	Euler's Theorem	10
4	The Chinese Remainder Theorem	12
4.1	What CRT Says: A Tiny Puzzle First	12
4.2	The Theorem and Its Proof	13
4.3	Solving CRT Problems by Hand: What a Problem Looks Like	14
4.4	Method A: Substitution (easiest by hand, no big formulas)	15

4.5	Method C: Describe x in two ways and compare	17
5	RSA: Construction, Correctness and Fast Decryption	20
5.1	Why Decryption Undoes Encryption (the Coprime Case)	20
5.2	Key Generation, Encryption and Decryption	21
5.3	Correctness of RSA	22
5.4	Fast Modular Exponentiation	23
5.5	CRT-Based Decryption Speedup	25
6	Four Attacks on RSA: Håstad, Common Modulus, Shared Primes, Fermat	28
6.1	Håstad's Broadcast Attack	28
6.2	Common Modulus Attack	30
6.3	Shared-Prime (Batch GCD) Attack	32
6.4	Fermat's Factorization (Close Primes)	34

1 Overview: The Problem RSA Solves, and the Order of the Tools

Before we define RSA, let's get one thing completely straight: *what problem is RSA actually trying to solve?* Everything in Sections 2–4 is in service of answering that one question, so we start there instead of jumping straight into definitions.

Ordinary encryption is easy to design if you only need to scramble a message once: shift every letter by 3, say. The hard part is **decryption** — undoing the scramble — and specifically, doing so *without* the two parties ever having met to agree on a shared secret in advance. RSA's answer is beautifully simple to state and much less simple to justify:

- To **encrypt** a message m (thought of as a number), raise it to some public power: $c = m^e$.
- To **decrypt**, raise the ciphertext to a different, secret power: c^d .

For this to work as a cryptosystem, we need one very specific mathematical guarantee:

$$(m^e)^d = m^{ed} \quad \text{must always equal } m.$$

That's it. That's the entire mathematical content RSA needs to be correct: *how do we choose e and d so that m^{ed} always comes back around to m ?* Ordinary arithmetic gives no such guarantee — m^{ed} just grows, it doesn't loop back to m . The trick is to do all of this arithmetic **modulo** some number n instead of over the ordinary integers, because modular arithmetic has a property ordinary arithmetic lacks: it *wraps around*, like a clock. A clock has only finitely many positions, so the powers $a, a^2, a^3, \dots$ of a number must eventually repeat a value, and once a value repeats the sequence cycles forever. Euler's theorem (Section 3) is the precise statement of *how far you have to raise a number's power before it wraps all the way back to where it started* — not just eventually, but at a specific, computable point. That is the punchline everything below builds toward; keep it in mind, because it is easy to get lost in definitions and forget what they are for.

The order of the sections. The sections are ordered so that each one uses only what came before it. Modular inverses and Bézout's identity (Section 2) come first because they are needed inside the proof of Euler's theorem (Section 3), inside the proof and the solution methods of the Chinese Remainder Theorem (Section 4), and to compute d from e . RSA itself (Section 5) comes only once Euler and CRT are available, because its correctness proof and its fast decryption use them; the attacks (Section 6) come last because each one is a tool from Sections 2–4 turned against a careless deployment.

2 Modular Arithmetic and the Extended Euclidean Algorithm

Why we need it. Modular inverses are needed three times in this lesson: to find the RSA private exponent d with $e \cdot d \equiv 1 \pmod{\varphi(n)}$ (Section 5), inside the proof of Euler’s theorem (Section 3), and to solve CRT systems (Section 4). For real RSA keys $\varphi(n)$ has hundreds of digits, so we cannot guess the inverse or try all values. The Extended Euclidean Algorithm (EEA) computes it in a few thousand simple steps, each just an integer division. This section covers: (0) a quick reminder of modular arithmetic, (1) the ordinary Euclidean algorithm (the gcd), (2) Bézout’s identity, (3) the table method, and (4) using it to get a modular inverse.

2.1 Modular Arithmetic in 60 Seconds

If you already know what “ $a \equiv b \pmod{n}$ ” means, skip ahead. If not: think of a clock face with n numbers on it, $0, 1, \dots, n-1$. Counting forward past $n-1$ wraps back around to 0. “ $a \bmod n$ ” just means: the position on this n -hour clock that you land on after counting a steps forward from 0. Two numbers a, b are **congruent mod n** , written $a \equiv b \pmod{n}$, if they land on the *same* clock position — equivalently, if n divides $a - b$ exactly. All of ordinary addition, subtraction, and multiplication still work perfectly well on the clock: you can add, subtract, or multiply two clock positions and it doesn’t matter whether you reduce mod n before or after doing the arithmetic, you land in the same place either way. This “wraps around and still behaves like normal arithmetic” property is the single fact that makes everything below possible.

2.2 The ordinary Euclidean algorithm

Lemma 2.1. *For integers $a \geq b > 0$, write $a = qb + r$ with $0 \leq r < b$ (division with remainder). Then $\gcd(a, b) = \gcd(b, r)$.*

What it says. Divide a by b and keep the remainder r . The new, smaller pair (b, r) has the same gcd as the old pair (a, b) . So we may replace (a, b) by (b, r) and repeat. The remainders strictly decrease, so we reach remainder 0; the last nonzero remainder is the gcd, because $\gcd(g, 0) = g$.

Example 2.2 ($\gcd(240, 46)$).

$$240 = 5 \cdot 46 + 10$$

$$46 = 4 \cdot 10 + 6$$

$$10 = 1 \cdot 6 + 4$$

$$6 = 1 \cdot 4 + 2$$

$$4 = 2 \cdot 2 + 0$$

The last nonzero remainder is 2, so $\gcd(240, 46) = 2$.

The number of division steps grows only in proportion to the number of digits of the inputs, so for 2048-bit numbers it is at most a few thousand steps.

2.3 Bézout’s identity: the gcd is a combination

Theorem 2.3 (Bézout). *For integers a, b (not both zero) there exist integers s, t with*

$$sa + tb = \gcd(a, b).$$

Look at Example 2.2: the first line says $10 = 240 - 5 \cdot 46$, which is 10 written as a combination of 240 and 46. The second line gives $6 = 46 - 4 \cdot 10$, and substituting the previous line turns this into another combination of 240 and 46. Every remainder we produce can be rewritten as a combination of the two original numbers, and the last nonzero remainder is the gcd. That is the whole proof, and it is also the algorithm: *keep track of the combination for each remainder as you go*. The table below does exactly this.

Lemma 2.4 (Coprime divisors multiply). *If $\gcd(m, n) = 1$ and both $m \mid z$ and $n \mid z$, then $mn \mid z$.*

Proof. By Theorem 2.3 there are integers s, t with $sm + tn = 1$. Multiplying by z gives $z = smz + tnz$. The first term is divisible by mn because $n \mid z$, and the second is divisible by mn because $m \mid z$. Hence $mn \mid z$. $\square$

This small fact is used later in two places: to show that a CRT solution is unique (Theorem 4.2), and to combine “divisible by p ” and “divisible by q ” into “divisible by pq ” in the RSA correctness proof (Theorem 5.7).

2.4 The table method

The recipe

Start with the two numbers $a \geq b$ you are given. Build a table with columns i, q_i, r_i, s_i, t_i .

Two starting rows (memorize these):

$$r_0 = a, \quad s_0 = 1, \quad t_0 = 0 \qquad r_1 = b, \quad s_1 = 0, \quad t_1 = 1.$$

Every next row uses the two rows just above it:

$$q_i = \left\lfloor \frac{r_{i-2}}{r_{i-1}} \right\rfloor, \quad r_i = r_{i-2} - q_i r_{i-1}, \quad s_i = s_{i-2} - q_i s_{i-1}, \quad t_i = t_{i-2} - q_i t_{i-1}.$$

Stop when a remainder $r_i = 0$. The row just before it holds $\gcd(a, b) = r_{i-1} = s_{i-1}a + t_{i-1}b$.

Notice that r, s, t all follow the *same* update rule with the same q_i ; only the starting values differ. So each new row is: compute q from the two r ’s, then do the same “previous-previous minus q times previous” in three columns.

The key invariant. In every row,

$$r_i = s_i a + t_i b$$

It holds for the starting rows ($a = 1 \cdot a + 0 \cdot b$ and $b = 0 \cdot a + 1 \cdot b$). If it holds for two consecutive rows, subtracting q_i times one equation from the other shows it holds for the next row. So it holds for every row, including the one with $r = \gcd$. *This is your built-in error check:* at any row you can verify $s_i a + t_i b = r_i$.

Example 2.5 (Table for $a = 240, b = 46$). Run the recipe with $a = 240$ and $b = 46$ (same numbers as Example 2.2):

i	q_i	r_i	s_i	t_i
0	—	240	1	0
1	—	46	0	1
2	5	10	1	−5
3	4	6	−4	21
4	1	4	5	−26
5	1	2	−9	47
6	2	0	23	−120

Row 5 is the last one before the remainder hits 0: $\gcd = 2$ and $2 = -9 \cdot 240 + 47 \cdot 46$. One row being computed: row 3 uses rows 1 and 2, so $q_3 = \lfloor 46/10 \rfloor = 4$, then $r_3 = 46 - 4 \cdot 10 = 6$, $s_3 = 0 - 4 \cdot 1 = -4$, $t_3 = 1 - 4 \cdot (-5) = 21$. Check the invariant: $-4 \cdot 240 + 21 \cdot 46 = -960 + 966 = 6$. ✓

(The final row's s, t , here 23 and -120 , equal $\pm b/g$ and $\mp a/g$; they are not used.)

2.5 From Bézout to the modular inverse

Definition 2.6 (Modular inverse). The *inverse* of x modulo m is a number y such that $x \cdot y$ leaves remainder 1 when divided by m , i.e. $xy \equiv 1 \pmod{m}$. We write $y \equiv x^{-1} \pmod{m}$. Theorem 2.7 below shows that it exists exactly when x and m share no common factor (this is why CRT needs coprime moduli).

Theorem 2.7. Let $m > 1$ and $\gcd(x, m) = 1$. Run the EEA on $(a, b) = (m, x)$. If the row with $r = 1$ is $1 = sm + tx$, then

$$x^{-1} \equiv t \pmod{m}.$$

If $\gcd(x, m) \neq 1$, no inverse exists.

Proof. Reduce $1 = sm + tx$ modulo m : the term sm vanishes, leaving $1 \equiv tx \pmod{m}$. That is exactly the definition of t being the inverse of x . If $g = \gcd(x, m) > 1$, then g divides $tx + sm$ for every s, t , so $tx \equiv 1$ is impossible. □

Practical shortcuts for inverses.

- Put the *modulus first*: $a = m$, $b = x$. Then t is the inverse and the s column is unnecessary — you may leave it out to save work.
- If t comes out negative, add m (repeat until it lies in $0, \dots, m-1$).

For small numbers you can skip the table. Replace x by its remainder modulo m , then compute $x, 2x, 3x, \dots$ (reducing modulo m) until you see 1; the multiplier that gives 1 is the inverse. This is fine by hand when m is small and hopeless when m has hundreds of digits.

Example 2.8 (Inverse of 7 modulo 9). $7 \cdot 1 = 7$, $7 \cdot 2 = 14 \equiv 5$, $7 \cdot 3 = 21 \equiv 3$, $7 \cdot 4 = 28 \equiv 1$. So $7^{-1} \equiv 4 \pmod{9}$.

Example 2.9 (Inverse of 35 modulo 4). First reduce: $35 = 8 \cdot 4 + 3$, so $35 \equiv 3$. Then $3 \cdot 1 = 3$, $3 \cdot 2 = 6 \equiv 2$, $3 \cdot 3 = 9 \equiv 1$. So $35^{-1} \equiv 3 \pmod{4}$.

Example 2.10 (Small, by hand: the inverse of 7 modulo 40). Run the EEA with the modulus first: $a = 40$, $b = 7$.

i	q_i	r_i	s_i	t_i
0	—	40	1	0
1	—	7	0	1
2	5	5	1	-5
3	1	2	-1	6
4	2	1	3	-17
5	2	0	-7	40

The highlighted row says $1 = 3 \cdot 40 + (-17) \cdot 7$. Hence $7^{-1} \equiv -17 \equiv 40 - 17 = \mathbf{23} \pmod{40}$.

Check: $7 \cdot 23 = 161 = 4 \cdot 40 + 1$. ✓

Example 2.11 (RSA: find d for $e = 17$ and $\varphi(n) = 3120$). Take the textbook key $p = 61$, $q = 53$: $n = 3233$ and $\varphi(n) = 60 \cdot 52 = 3120$. We need $d = 17^{-1} \pmod{3120}$. (RSA itself is defined in Section 5; for now this is simply an inverse computation.) First confirm $\gcd(17, 3120) = 1$; the table will do that for us. Run the EEA with $a = 3120$ and $b = 17$:

i	q_i	r_i	s_i	t_i
0	—	3120	1	0
1	—	17	0	1
2	183	9	1	-183
3	1	8	-1	184
4	1	1	2	-367
5	8	0	-17	3120

Reading the table, step by step.

- Row 2: $3120 = 183 \cdot 17 + 9$, so $q_2 = 183$, $r_2 = 9$; $s_2 = 1 - 183 \cdot 0 = 1$; $t_2 = 0 - 183 \cdot 1 = -183$.
- Row 3: $17 = 1 \cdot 9 + 8$, so $q_3 = 1$, $r_3 = 8$; $s_3 = 0 - 1 \cdot 1 = -1$; $t_3 = 1 - 1 \cdot (-183) = 184$.
- Row 4: $9 = 1 \cdot 8 + 1$, so $q_4 = 1$, $r_4 = 1$; $s_4 = 1 - 1 \cdot (-1) = 2$; $t_4 = -183 - 1 \cdot 184 = -367$.
- Row 5: $8 = 8 \cdot 1 + 0$. The remainder is 0, so stop; the row above (row 4) has $r = 1 = \gcd$.

Row 4 gives $1 = 2 \cdot 3120 + (-367) \cdot 17$. Check: $6240 - 6239 = 1$. ✓

So $17^{-1} \equiv -367 \pmod{3120}$, and adding 3120 gives

$$d = 3120 - 367 = \mathbf{2753}.$$

Check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$. ✓

Example 2.12 (When there is no inverse: 18 modulo 60). Run the EEA with $a = 60$ and $b = 18$:

i	q_i	r_i	s_i	t_i
0	—	60	1	0
1	—	18	0	1
2	3	6	1	-3
3	3	0	-3	10

The last nonzero remainder is $6 \neq 1$, so $\gcd(18, 60) = 6$ and 18 has no inverse mod 60. This is why RSA requires $\gcd(e, \varphi(n)) = 1$: if the table ends with a gcd bigger than 1, choose a different e .

Common pitfall

- **Order matters for reading t .** With $(a, b) = (m, x)$ the inverse of x is the t column. If you swap the order, the inverse of x is in the s column instead.
- **Stop at the right row.** The answer is in the row *before* the remainder 0, not in the row where $r = 0$.
- **Negative answers are fine.** $t = -367$ means $-367 \equiv 2753 \pmod{3120}$; add the modulus.
- **Always verify.** Multiply your inverse by x and reduce; you must get 1.

Where else this appears. The same computation is used, with both s and t , in the *Common Modulus Attack* (Section 6.2): with $\gcd(e_1, e_2) = 1$ the EEA gives a, b with $ae_1 + be_2 = 1$, letting an attacker combine two ciphertexts into the plaintext. In the *Shared-Prime Attack* (Section 6.3) the plain Euclidean algorithm (Section 2.2) finds the common prime factor of two moduli. In RSA key generation (Section 5.2), the EEA is the step that turns the public exponent e into the private exponent d .

3 Euler's Totient Function, Euler's Theorem and Fermat's Little Theorem

This section proves the fact RSA is built on: if a shares no factor with n , then raising a to the power $\varphi(n)$ wraps all the way back to 1 modulo n (the “how far must a power go?” question from Section 1). We first define $\varphi(n)$, then show that the residues it counts form a self-contained system in which every element can be undone, and then prove the theorem.

3.1 Euler's Totient Function: Counting the “Well-Behaved” Numbers

Here's the question that gets us there. Fix a modulus n . Some numbers, when you keep multiplying them together mod n , behave completely predictably and reversibly — you can always undo a multiplication by them. Other numbers behave badly and lose information once you multiply by them mod n (think of multiplying by an even number mod an even n : you can never fully recover what you multiplied). **Euler's totient function**, written $\varphi(n)$, simply *counts how many numbers in $\{1, \dots, n\}$ are the “well-behaved,” reversible kind*. The formal condition for being “well-behaved” turns out to be exactly “sharing no common factor with n ” — we'll see why in a moment — so that becomes the definition:

Definition 3.1 (Euler's totient function). For $n \in \mathbb{Z}^+$, define

$$\varphi(n) = |\{a \in \{1, \dots, n\} : \gcd(a, n) = 1\}|,$$

i.e. $\varphi(n)$ counts how many integers in $\{1, \dots, n\}$ share *no* common factor with n other than 1 (such integers are called **coprime to n** , or **relatively prime to n**).

Example 3.2 (Computing φ by brute force, to build intuition). Take $n = 10$. Check every integer from 1 to 10 against $\gcd(\cdot, 10)$:

a	1	2	3	4	5	6	7	8	9	10
$\gcd(a, 10)$	1	2	1	2	5	2	1	2	1	10

The values with $\gcd(a, 10) = 1$ are $\{1, 3, 7, 9\}$ — exactly 4 of them, so $\varphi(10) = 4$. Notice $\varphi(10)$ is not simply “10 minus something obvious”; it depends on *which* primes divide 10 (here 2 and 5), not just how many divisors 10 has.

Proposition 3.3 (Formula for φ). If $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ is the prime factorization of n , then

$$\varphi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right).$$

In particular, for two distinct primes p, q :

$$\varphi(pq) = (p-1)(q-1).$$

This special case is the one RSA uses directly.

Remark 3.4 (Where this formula comes from, intuitively). Think inclusion–exclusion. Among any p_1 consecutive integers, exactly one is divisible by p_1 , so a $\left(1 - \frac{1}{p_1}\right)$ fraction survives once multiples of p_1 are discarded. The same holds independently for $p_2, \dots, p_k$, and because the p_i are distinct primes, being “divisible by p_i ” and being “divisible by p_j ” turn out not to interfere with each other across

$\{1, \dots, n\}$, so the surviving fraction after excluding every prime factor is the *product* of the individual surviving fractions. Multiplying by n gives the formula. The RSA-relevant case $\varphi(pq) = (p-1)(q-1)$ says the same thing very concretely: from $\{1, \dots, pq\}$, throw away the q multiples of p and the p multiples of q (these two lists overlap only at pq itself, discarded either way), leaving $pq - p - q + 1 = (p-1)(q-1)$ survivors.

Example 3.5 (Computing φ with the formula). $36 = 2^2 \cdot 3^2$, so $\varphi(36) = 36 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{3}\right) = 36 \cdot \frac{1}{2} \cdot \frac{2}{3} = 12$. Direct check: $\{1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35\}$ are exactly the 12 residues coprime to 36 — none of them divisible by 2 or by 3.

For the RSA-relevant case: $p = 61, q = 53$ gives $\varphi(61 \cdot 53) = 60 \cdot 52 = 3120$ — this is the pair we will reuse throughout this lecture.

3.2 Why We Care: The Coprime Numbers Form a Self-Contained “Club”

Here is the fact that makes $\varphi(n)$ worth naming at all, explained slowly, with no jargon first, and then with the standard vocabulary attached afterward.

Claim: if you take two numbers that are each coprime to n , and multiply them mod n , the result is *again* coprime to n . In words: multiplying two “well-behaved” numbers mod n never produces a “badly-behaved” one. This is not obvious from nothing, but it is easy to see why it’s true: if a and n share no prime factor, and b and n share no prime factor, then ab cannot suddenly acquire a prime factor of n out of nowhere either — multiplication doesn’t create new prime factors that weren’t already in a or b . So the set of numbers coprime to n is **closed** under multiplication mod n : start inside the set, multiply, you stay inside the set.

That alone would already be interesting, but there is more: every number coprime to n also has a genuine *multiplicative inverse* mod n — another number in the same set that, when multiplied by it, gives 1. (By Theorem 2.7 such an inverse exists precisely because $\gcd(a, n) = 1$, and the Extended Euclidean Algorithm computes it.) So this set of “well-behaved” numbers is not just closed under multiplication — every element can also be *undone*.

A quick note on the word “group.” Mathematicians have a name for exactly this kind of structure: a set with an operation (here, multiplication mod n) that (i) is closed (combining two elements of the set stays in the set), (ii) has an identity element (here, 1, which does nothing when multiplied), and (iii) gives every element an inverse within the set. Such a structure is called a **group**. You do not need to know any general group theory to follow this lecture — we will only ever use the three properties just described, applied to this one specific set. We just want a name for it, because we will refer to it constantly (for $n \geq 2$):

$$(\mathbb{Z}/n\mathbb{Z})^* = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1\},$$

the set of “well-behaved,” invertible residues mod n . By definition, this set has exactly $\varphi(n)$ elements. **Euler’s theorem, stated next, is a fact about exactly this set.**

3.3 Euler’s Theorem

Theorem 3.6 (Euler’s Theorem). *If $\gcd(a, n) = 1$, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Plain-English statement, before the proof. Take any a sharing no factor with n — i.e. any a in our “well-behaved” club $(\mathbb{Z}/n\mathbb{Z})^*$ — and start computing $a^1, a^2, a^3, \dots$ modulo n . Euler’s theorem

guarantees that by the time you reach exponent $\varphi(n)$, you have landed back exactly on 1 — regardless of which coprime a you started with, and regardless of how large or small n is. This “guaranteed return to 1 after exactly $\varphi(n)$ steps” is the single fact RSA is built on: it is precisely what lets a carefully chosen *second* exponentiation undo the effect of a first one, which is the problem posed in Section 1 and solved in Section 5.1.

Proof — the constructive version, worth doing by hand once. We prove it by directly multiplying every element of the group $(\mathbb{Z}/n\mathbb{Z})^* = \{g_1, \dots, g_{\varphi(n)}\}$ by a and watching what happens, rather than citing an abstract theorem.

Step 1: multiplying by a just reorders the group. Consider $ag_1, ag_2, \dots, ag_{\varphi(n)} \pmod{n}$. Each ag_i is still coprime to n (a product of two things coprime to n , by the closure property discussed above), so every value in this new list is again some element of $(\mathbb{Z}/n\mathbb{Z})^*$. No two of them coincide either: if $ag_i \equiv ag_j \pmod{n}$, then since $\gcd(a, n) = 1$ we may multiply both sides by $a^{-1} \pmod{n}$ (which exists by Theorem 2.7, precisely because $\gcd(a, n) = 1$) to get $g_i \equiv g_j \pmod{n}$, forcing $i = j$. So the list $ag_1, \dots, ag_{\varphi(n)}$ consists of $\varphi(n)$ *distinct* elements, all drawn from a set that only has $\varphi(n)$ elements total — which means it must simply be $\{g_1, \dots, g_{\varphi(n)}\}$, written out in a different order.

Step 2: multiply everything together and cancel. Both lists contain exactly the same elements (just reordered), so their products mod n agree:

$$(ag_1)(ag_2) \cdots (ag_{\varphi(n)}) \equiv g_1 g_2 \cdots g_{\varphi(n)} \pmod{n}.$$

The left side equals $a^{\varphi(n)} \cdot P$, where $P = g_1 g_2 \cdots g_{\varphi(n)}$. Since P is a product of things coprime to n , $\gcd(P, n) = 1$, so P has an inverse mod n . The equation $a^{\varphi(n)} \cdot P \equiv P \pmod{n}$ then gives, after multiplying both sides by $P^{-1} \pmod{n}$,

$$a^{\varphi(n)} \equiv 1 \pmod{n}. \quad \square$$

Remark 3.7 (The one-line abstract version, for readers who already know group theory). The argument above *is* a proof of Lagrange’s theorem specialized to this one group: $(\mathbb{Z}/n\mathbb{Z})^*$ is a finite group of order $\varphi(n)$, and for any element a of any finite group G , $a^{|G|}$ equals the identity. The longer proof above is the same idea, unpacked into elementary steps that use only “multiplying reorders a finite set” and “cancel a common invertible factor.”

Example 3.8 (Euler’s theorem, traced by hand). Let $n = 10$, so $(\mathbb{Z}/10\mathbb{Z})^* = \{1, 3, 7, 9\}$ and $\varphi(10) = 4$. Take $a = 3$ and multiply every group element by 3, mod 10:

$$3 \cdot 1 = 3, \quad 3 \cdot 3 = 9, \quad 3 \cdot 7 = 21 \equiv 1, \quad 3 \cdot 9 = 27 \equiv 7.$$

The results $\{3, 9, 1, 7\}$ are exactly $\{1, 3, 7, 9\}$ again, just shuffled — precisely Step 1 of the proof. Multiplying the original four elements together: $P = 1 \cdot 3 \cdot 7 \cdot 9 = 189$. The proof says $3^4 \cdot P \equiv P \pmod{10}$, i.e. $3^4 \equiv 1 \pmod{10}$ after cancelling P . Direct check: $3^4 = 81 = 8 \cdot 10 + 1 \equiv 1 \pmod{10}$.
✓

Corollary 3.9 (Fermat’s Little Theorem). *If p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$. This is Euler’s theorem with $n = p$, since $\varphi(p) = p - 1$ (every integer from 1 to $p - 1$ is automatically coprime to a prime p).*

4 The Chinese Remainder Theorem

The Chinese Remainder Theorem (CRT) says that knowing a number’s remainders modulo two co-prime numbers is exactly as good as knowing its remainder modulo their product. RSA uses it twice (Section 5.3: a correctness proof for *every* message; Section 5.5: faster decryption), and Håstad’s attack (Section 6.1) is built on it. This section first explains what CRT says and proves it (Sections 4.1 and 4.2), and then shows how to solve CRT problems by hand with two methods: **Method A** (substitution) and **Method C** (describe x in two ways and compare).

4.1 What CRT Says: A Tiny Puzzle First

Step 1: the basic idea, with a tiny puzzle

Suppose you’re told a number x leaves remainder 2 when divided by 3, and remainder 3 when divided by 5. In symbols,

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}.$$

Is there such a number? Is it the only one? CRT exists to answer exactly this kind of question.

Step 2: solve it by brute force first

Before any formula, just think it through. Numbers leaving remainder 2 when divided by 3 are

$$2, 5, 8, 11, 14, 17, 20, \dots$$

Now check each one against the second condition (remainder 3 when divided by 5):

$$2 \div 5 \rightarrow \text{rem } 2 \ (\times), \quad 5 \div 5 \rightarrow \text{rem } 0 \ (\times), \quad 8 \div 5 \rightarrow \text{rem } 3 \ (\checkmark).$$

So $x = 8$ works. And so does 23, and 38, and 53 — every number 15 apart from 8, because $3 \times 5 = 15$. So the full answer isn’t just “ $x = 8$,” it’s

$$x \equiv 8 \pmod{15}.$$

That’s the pattern CRT formalizes: two remainder conditions, on moduli sharing no common factor, pin down x uniquely *once you fix the modulus* $15 = 3 \times 5$ — not a single number, but a single residue class mod 15. (A practical tip: list the numbers for the *bigger* modulus, here 3, 8, 13, $\dots$; adding 5 cycles through all remainders mod 3, so you never need more than 3 candidates.)

Step 3: the general statement, in words

If you know x ’s remainder mod m and x ’s remainder mod n , where m and n share no common factor, that is *exactly as much information* as knowing x ’s remainder mod mn directly. Nothing is lost by splitting the modulus, and nothing is ambiguous: every pair of remainders corresponds to one and only one $x \bmod mn$. This is the fact RSA is built on: with m and n the two primes p, q of an RSA modulus, it lets us trade one hard computation mod pq for two easier computations, mod p and mod q separately, with a guarantee that stitching the two answers back together recovers the truth mod pq .

Remark 4.1 (A mental picture: two interlocking gears). Picture two gears with m and n teeth, both starting at a marked tooth and turning together. If m and n share no common factor, the combined picture — “which tooth of gear 1 is marked now, which tooth of gear 2 is marked now” — does not repeat until mn turns have passed, and every one of the mn possible combinations occurs exactly once per cycle. “Remainder mod m ” and “remainder mod n ” are the two gears’ positions; CRT says these two positions, taken together, pin down the combined state uniquely — the same information as one remainder mod mn .

4.2 The Theorem and Its Proof

Theorem 4.2 (CRT). *Let $m, n \geq 2$ be integers with $\gcd(m, n) = 1$. The map*

$$\mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \quad x \mapsto (x \bmod m, x \bmod n)$$

is a ring isomorphism. Equivalently: for every pair (a, b) the system

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has exactly one solution $x \in \{0, \dots, mn - 1\}$; all other integer solutions differ from it by multiples of mn .

Unpacking “ring isomorphism” one word at a time. “Map” just means: send x to the pair of its two remainders — exactly what we did by hand in Step 2. “Isomorphism” means this map is a perfect one-to-one correspondence between $\{0, \dots, mn - 1\}$ and all pairs (a, b) — this is the “unique solution” half of the theorem, and it’s the part the puzzle in Steps 1–2 already illustrated concretely. “Ring” (rather than just “set”) adds one more guarantee: the correspondence also respects *arithmetic*. Adding (or multiplying) two numbers mod mn and then splitting into remainders gives the same answer as splitting first and adding (or multiplying) each coordinate separately:

$$x + y \mapsto (a + c \bmod m, b + d \bmod n), \quad xy \mapsto (ac \bmod m, bd \bmod n) \quad \text{when } x \mapsto (a, b), y \mapsto (c, d).$$

This extra guarantee is what makes CRT more than a counting trick: an entire arithmetic computation can be carried out *coordinatewise*, mod m and mod n separately, and the two partial answers automatically recombine correctly mod mn — exactly the mechanism Section 5.3 uses for a correctness proof covering every message (not just the coprime ones) and Section 5.5 uses for faster decryption.

Proof, explained step by step. We check three things: the map preserves arithmetic, it never sends two different numbers to the same pair, and — by a counting argument — it must therefore hit every pair.

It preserves arithmetic. If $x \equiv a \pmod{m}$ and $y \equiv c \pmod{m}$, then directly from the definition of congruence, $x + y \equiv a + c \pmod{m}$ and $xy \equiv ac \pmod{m}$ — and likewise mod n . So reducing mod m (or mod n) *after* adding/multiplying gives the same result as adding/multiplying the already-reduced remainders. That’s exactly the coordinatewise formula above.

It is injective (no two different numbers share a remainder pair). Suppose x and y , both in $\{0, \dots, mn - 1\}$, map to the *same* pair: $x \equiv y \pmod{m}$ and $x \equiv y \pmod{n}$. The first says $m \mid (x - y)$; the second says $n \mid (x - y)$. Since $\gcd(m, n) = 1$, Lemma 2.4 gives $mn \mid (x - y)$. But $|x - y| < mn$, and the only multiple of mn smaller than mn in absolute value is 0 — so $x = y$. Distinct inputs always land on distinct output pairs.

It is therefore also surjective (every pair is hit). The domain $\{0, \dots, mn - 1\}$ has exactly mn elements. The codomain — all pairs (a, b) with $0 \leq a < m, 0 \leq b < n$ — also has exactly $m \cdot n$ elements. An injective map between two finite sets of the *same size* must also be surjective: if some pair were missed, the mn distinct images would have to squeeze into fewer than mn remaining slots, which is impossible. So every pair (a, b) is the image of exactly one x — the “unique solution” claim. $\square$

The proof shows that a solution exists and is unique, but it does not hand us one. Methods A and C below construct it by hand; Theorem 4.6 is the closed formula.

4.3 Solving CRT Problems by Hand: What a Problem Looks Like

Theorem 4.2 says a solution exists and is unique; here we learn *how to compute it* with two methods: **Method A** (substitution) and **Method C** (describe x in two ways and compare). Method C is also the method used in practice for RSA decryption (Section 5.5), and it is the tool used in Håstad's attack (Section 6.1). You do not need any theory to use the recipes: every step is explained with actual numbers.

You are told the *remainders* of an unknown whole number x when it is divided by several numbers, and you must find x :

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \dots, \quad x \equiv a_k \pmod{m_k}.$$

Step 0: check the moduli. Look at every *pair* of moduli. If no two of them share a common factor larger than 1, they are called *pairwise coprime* and everything below works directly.

Moduli	Verdict
3, 5, 7	pairwise coprime
4, 9, 25	pairwise coprime (no shared prime factor)
6, 9	<i>not</i> coprime (both divisible by 3): the recipes below do not apply directly

What the answer looks like. Let $M = m_1 m_2 \cdots m_k$ be the product of the moduli. Then there is *exactly one* solution between 0 and $M - 1$, and every other solution is that one plus or minus a multiple of M . We write the complete answer as

$$x \equiv x_0 \pmod{M}.$$

For example, for the moduli 3, 5, 7 we have $M = 105$, and the solutions are $\dots, -82, 23, 128, 233, \dots$ — all of them differ by multiples of 105. (For two moduli this is Theorem 4.2; for more, the recipes below merge two congruences at a time, and each merge produces one congruence modulo the product of the two moduli, which is still coprime to the remaining ones.)

4.4 Method A: Substitution (easiest by hand, no big formulas)

Recipe for Method A

1. Take one congruence, say $x \equiv a \pmod{m}$, and rewrite it as an *equation*:

$$x = a + mt \quad (t \text{ is an unknown whole number}).$$

2. Put this into the next congruence $x \equiv b \pmod{n}$. You get $a + mt \equiv b \pmod{n}$, i.e.

$$mt \equiv b - a \pmod{n}.$$

3. Solve this small congruence for t : you get $t \equiv t_0 \pmod{n}$. (Try values of $t = 0, 1, 2, \dots, n-1$, or multiply by an inverse, see Section 2.5.)
4. Write $t = t_0 + ns$ and put it back:

$$x = a + m(t_0 + ns) = \underbrace{(a + mt_0)}_{\text{new } a} + \underbrace{mn}_{\text{new } m} s.$$

So the two congruences have been merged into the single congruence $x \equiv a + mt_0 \pmod{mn}$.

5. Repeat with the next congruence until none are left.

Tip: it is least work to start with the **largest** modulus.

Example 4.3 (Method A in symbols, same problem). Solve $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$ with the recipe.

Step 1. Start from the modulus 5: $x = 3 + 5t$.

Step 2. Put into the other congruence: $3 + 5t \equiv 2 \pmod{3}$. Subtract 3: $5t \equiv -1 \pmod{3}$.

Step 3. Solve $5t \equiv -1 \pmod{3}$ for t . We do it in three small moves.

- Make the left number small.** Dividing 5 by 3 leaves remainder 2, so $5 \equiv 2 \pmod{3}$. Hence $5t$ behaves like $2t$, and the congruence becomes $2t \equiv -1 \pmod{3}$.
- Make the right number positive.** Adding the modulus 3 does not change a remainder, so $-1 \equiv -1 + 3 = 2 \pmod{3}$. The congruence becomes $2t \equiv 2 \pmod{3}$.
- Find t .** Ask: which t makes $2t$ leave remainder 2 when divided by 3? Just try the possible remainders $t = 0, 1, 2$:

t	0	1	2
$2t$	0	2	4
remainder of $2t$ divided by 3	0	2	1
equals 2?	no	yes	no

Only $t = 1$ works, so $t \equiv 1 \pmod{3}$, i.e. $t \in \{1, 4, 7, \dots\}$.

(Shortcut: since 2 and 3 share no factor, you may divide both sides of $2t \equiv 2$ by 2 and get $t \equiv 1$. The table above shows this is safe.)

Step 4. Write $t = 1 + 3s$: $x = 3 + 5(1 + 3s) = 8 + 15s$.

Answer: $x \equiv 8 \pmod{15}$.

Example 4.4 (Sunzi's riddle by Method A: three congruences). Solve

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

Here $M = 3 \cdot 5 \cdot 7 = 105$. Start with the largest modulus, 7.

Round 1: bring in the modulus 5.

$$\begin{array}{ll} x = 2 + 7t & \text{(from } x \equiv 2 \pmod{7} \text{)} \\ 2 + 7t \equiv 3 \pmod{5} & \text{(the second condition)} \\ 7t \equiv 1 \pmod{5} & \text{(subtract 2)} \\ 2t \equiv 1 \pmod{5} & (7 \equiv 2 \pmod{5}) \\ t \equiv 3 \pmod{5} & \text{(because } 2 \cdot 3 = 6 \equiv 1 \text{)} \end{array}$$

So $t = 3 + 5s$ and $x = 2 + 7(3 + 5s) = 23 + 35s$. We have merged two conditions:

$$x \equiv 23 \pmod{35}.$$

Round 2: bring in the modulus 3.

$$\begin{array}{ll} x = 23 + 35s & \\ 23 + 35s \equiv 2 \pmod{3} & \text{(the last condition)} \\ 2 + 2s \equiv 2 \pmod{3} & (23 \equiv 2, 35 \equiv 2 \pmod{3}) \\ 2s \equiv 0 \pmod{3} & \\ s \equiv 0 \pmod{3} & \end{array}$$

So $s = 3r$ and $x = 23 + 35 \cdot 3r = 23 + 105r$.

Answer: $x \equiv 23 \pmod{105}$. The smallest positive solution is $x = 23$.

Check: $23 = 7 \cdot 3 + 2 \checkmark$; $23 = 4 \cdot 5 + 3 \checkmark$; $23 = 3 \cdot 7 + 2 \checkmark$

Example 4.5 (A harder one: moduli 4, 9, 25). Solve $x \equiv 1 \pmod{4}$, $x \equiv 2 \pmod{9}$, $x \equiv 3 \pmod{25}$.

Here $M = 4 \cdot 9 \cdot 25 = 900$.

Round 1. Start with 25: $x = 3 + 25t$. Put into the condition mod 9:

$$\begin{array}{ll} 3 + 25t \equiv 2 \pmod{9} & \\ 25t \equiv -1 \pmod{9} & \\ 7t \equiv 8 \pmod{9} & (25 \equiv 7, -1 \equiv 8) \end{array}$$

We need the inverse of 7 modulo 9, which is 4 (Example 2.8). So multiply both sides by 4:

$$t \equiv 8 \cdot 4 = 32 \equiv 5 \pmod{9}.$$

Thus $t = 5 + 9s$, and $x = 3 + 25(5 + 9s) = 128 + 225s$. So $x \equiv 128 \pmod{225}$.

Round 2. Put $x = 128 + 225s$ into the condition mod 4:

$$\begin{array}{ll} 128 + 225s \equiv 1 \pmod{4} & \\ 0 + 1 \cdot s \equiv 1 \pmod{4} & (128 \equiv 0, 225 \equiv 1 \pmod{4}) \\ s \equiv 1 \pmod{4} & \end{array}$$

So $s = 1 + 4r$ and $x = 128 + 225(1 + 4r) = 353 + 900r$.

Answer: $x \equiv 353 \pmod{900}$.

Check: $353 = 88 \cdot 4 + 1 \checkmark$; $353 = 39 \cdot 9 + 2 \checkmark$; $353 = 14 \cdot 25 + 3 \checkmark$

4.5 Method C: Describe x in two ways and compare

Method C solves **two** congruences

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}, \quad \gcd(m, n) = 1.$$

There is no formula to memorize: the answer comes out of one simple idea. (This is also the method used in practice, for instance in RSA decryption, because all its numbers stay small.)

The idea: “start at the remainder, then take steps”

What does $x \equiv a \pmod{m}$ say? That x is a plus some number of whole steps of size m :

$$x = a + m \cdot t \quad (t = 0, 1, 2, \dots \text{ counts the steps}).$$

For example, “remainder 2 when divided by 3” means $x \in \{2, 2+3, 2+6, 2+9, \dots\} = \{2, 5, 8, 11, \dots\}$.

In exactly the same way, $x \equiv b \pmod{n}$ says that x is b plus some number of steps of size n :

$$x = b + n \cdot s.$$

But x is *one* number, so both descriptions must give the same value:

$$a + m t = b + n s.$$

Our only unknowns are the step counts t and s . Now comes the one clever move: **look at both sides through the eyes of n** , i.e. look only at remainders when dividing by n . Then $n s$ is a multiple of n and simply *disappears*, leaving

$$a + m t \equiv b \pmod{n} \quad \Longleftrightarrow \quad m t \equiv b - a \pmod{n}.$$

In words: *the steps of size m must add up to exactly the gap $b - a$, as seen by n* . Find t from this, and the answer is $x = a + m t$.

Guided example: $x \equiv 2 \pmod{3}$ **and** $x \equiv 3 \pmod{5}$

Here $a = 2$, $m = 3$, $b = 3$, $n = 5$. Every number of the form $x = 2 + 3t$ already has remainder 2 when divided by 3, so we only have to choose the number of steps t that gives remainder 3 when divided by 5. That is exactly the listing of Step 2 in Section 4.1, now with the step count attached: $t = 0, 1, 2$ give $x = 2, 5, 8$, with remainders 2, 0, 3 modulo 5, so $t = 2$ and $x = 2 + 3 \cdot 2 = 8$. Going on, $t = 3, 4$ give remainders 1, 4, so the remainders 2, 0, 3, 1, 4 contain every value from 0 to 4 exactly once: since 3 and 5 share no factor, the steps of size 3 visit every remainder mod 5, so exactly one t between 0 and 4 works. Since the pattern repeats after $3 \cdot 5 = 15$, the full answer is $x \equiv 8 \pmod{15}$.

When the numbers are big: let the inverse find t for you

Trying $t = 0, 1, 2, \dots$ is fine for tiny numbers but hopeless for large ones. Look again at what we had to solve:

$$m t \equiv b - a \pmod{n} \quad (\text{above: } 3t \equiv 1 \pmod{5}).$$

If we had a number that “undoes” multiplication by m , we could just apply it to both sides. That number is the *inverse* m^{-1} of m modulo n : it satisfies $m \cdot m^{-1} \equiv 1 \pmod{n}$ (see Section 2.5 for how to find it). Multiplying both sides by it gives t directly:

$$t \equiv (b - a) \cdot m^{-1} \pmod{n}, \quad \text{and then} \quad x = a + m t \pmod{mn}.$$

In the example, $3 \cdot 2 = 6 \equiv 1 \pmod{5}$, so $3^{-1} = 2$ and $t \equiv (3 - 2) \cdot 2 = 2$. It is the same $t = 2$ as in the table, obtained without trying anything.

Why the answer works, in two lines. Modulo m : $x = a + mt$ differs from a by a multiple of m , so the remainder is a , whatever t is. Modulo n : $mt \equiv (b - a)mm^{-1} \equiv b - a$, so $x \equiv a + (b - a) = b$.

The formula, stated formally

Theorem 4.6 (Two-congruence formula (Method C)). *Let $m, n \geq 2$ be integers with $\gcd(m, n) = 1$, and let a, b be any integers. Let u be an inverse of m modulo n , i.e. an integer with $mu \equiv 1 \pmod{n}$. Then the system*

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has exactly one solution modulo mn , namely

$$x \equiv a + mt \pmod{mn}, \quad \text{where } t \equiv (b - a)u \pmod{n}.$$

Proof. It is a solution. Modulo m : $x - a = mt$ is a multiple of m , so $x \equiv a$. Modulo n : $mt \equiv mu(b - a) \equiv b - a$, so $x \equiv a + (b - a) = b$.

It is the only one. By the uniqueness part of Theorem 4.2, any other solution is congruent to this one modulo mn . $\square$

Symbols at a glance.

Symbol	Meaning
a, m	remainder and modulus of the first congruence
b, n	remainder and modulus of the second congruence
$u = m^{-1} \pmod{n}$	the number with $mu \equiv 1 \pmod{n}$
$b - a$	the gap the steps of size m must cover, as seen by n
t	the number of steps of size m taken from a
mn	the modulus of the final answer

Example 4.7 (Three congruences: Sunzi again, one merge at a time). Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

First merge the first two. We just found $x \equiv 8 \pmod{15}$. This single congruence replaces the first two.

Now merge it with the third. Take $a = 8$, $m = 15$ (so $x = 8 + 15t$) and $b = 2$, $n = 7$. We need $8 + 15t \equiv 2 \pmod{7}$, i.e. $15t \equiv -6 \pmod{7}$. Since $15 = 2 \cdot 7 + 1$, we have $15 \equiv 1$, so this is simply $t \equiv -6 \equiv 1 \pmod{7}$.

Hence $x = 8 + 15 \cdot 1 = 23$, and $x \equiv 23 \pmod{105}$. This agrees with Method A (Example 4.4).

Common slip	Correct version
Inverting m modulo m (impossible)	invert m modulo the <i>other</i> modulus n
Forgetting to reduce t	reduce t modulo n before computing $a + mt$
Reporting the answer modulo m or n	the answer is modulo mn

Remark 4.8 (Beyond two moduli: the general formula, if you've seen it elsewhere). Theorem 4.2 and Method C use exactly two moduli, since that is all RSA needs. But CRT works for any number of pairwise-coprime moduli $n_1, \dots, n_k$, and it's worth seeing the general pattern once so the two-modulus case above looks like a special case rather than a different theorem. Given

$$x \equiv a_1 \pmod{n_1}, \quad x \equiv a_2 \pmod{n_2}, \quad \dots, \quad x \equiv a_k \pmod{n_k},$$

set $N = n_1 n_2 \cdots n_k$, and for each i let $N_i = N/n_i$ (the product of all moduli *except* n_i) and let M_i be the inverse of $N_i \bmod n_i$, i.e. $N_i M_i \equiv 1 \pmod{n_i}$. Then

$$x \equiv \sum_{i=1}^k a_i N_i M_i \pmod{N}.$$

Why this works: each term $a_j N_j M_j$ is built so that $\bmod n_i$ it vanishes for every $j \neq i$ (because N_j is a multiple of n_i whenever $j \neq i$) and reduces to exactly a_i when $j = i$ (because $N_i M_i \equiv 1 \pmod{n_i}$ by construction) — so the whole sum matches $a_i \bmod n_i$, for every i simultaneously. Re-deriving the Step-1 puzzle this way: $N = 15$, $N_1 = 5$, $N_2 = 3$; $M_1 = 5^{-1} \bmod 3 = 2$ and $M_2 = 3^{-1} \bmod 5 = 2$; so $x \equiv 2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2 = 20 + 18 = 38 \equiv 8 \pmod{15}$ — the same $x = 8$ as before. (For $k = 2$ this reduces to the two-modulus formula of Theorem 4.6, rearranged into a more convenient form.)

5 RSA: Construction, Correctness and Fast Decryption

We now have every tool. Section 1 reduced RSA to one requirement, $m^{ed} \equiv m \pmod{n}$. Euclid (Section 2) lets us compute d from e , Euler's theorem (Section 3) makes the identity true for messages coprime to n , and CRT (Section 4) extends it to every message. This section first shows *why* decryption undoes encryption (Section 5.1), then turns that into the key-generation recipe (Section 5.2), proves correctness for every message (Section 5.3), and finally covers how RSA is computed efficiently: fast exponentiation (Section 5.4) and CRT decryption (Section 5.5).

5.1 Why Decryption Undoes Encryption (the Coprime Case)

Recall the goal from Section 1: we need $m^{ed} \equiv m \pmod{n}$ to hold, for suitably chosen e and d . Euler's theorem (Theorem 3.6) is exactly the tool needed. Suppose e and d are chosen so that

$$ed \equiv 1 \pmod{\varphi(n)}.$$

Read this condition in words first: it says ed , divided by $\varphi(n)$, leaves remainder 1 — equivalently, ed is *one more than some whole number of multiples of $\varphi(n)$* . Call that whole number of multiples k , so as an equation over the integers (not just modulo $\varphi(n)$):

$$ed = 1 + k\varphi(n), \quad \text{for some integer } k \geq 0.$$

Substitute this into m^{ed} and simplify one step at a time, assuming for now $\gcd(m, n) = 1$:

$$m^{ed} = m^{1+k\varphi(n)} = m^1 \cdot m^{k\varphi(n)} = m \cdot \left(m^{\varphi(n)}\right)^k.$$

The first equality just substitutes $ed = 1 + k\varphi(n)$; the second splits the exponent's sum into a product of powers (the exponent law $x^{s+t} = x^s x^t$); the third regroupes $m^{k\varphi(n)} = (m^{\varphi(n)})^k$ (the exponent law $x^{st} = (x^s)^t$). Now Euler's theorem enters: since $\gcd(m, n) = 1$, $m^{\varphi(n)} \equiv 1 \pmod{n}$. Substituting,

$$m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k \equiv m \pmod{n}.$$

Chaining the two displayed calculations together gives exactly $m^{ed} \equiv m \pmod{n}$ — decryption undoes encryption, which is exactly the problem we set out to solve.

Remark 5.1 (The “clock” intuition for the same argument, without the algebra). Picture the powers of m modulo n arranged around a dial with $\varphi(n)$ positions. Euler's theorem says raising m to the power $\varphi(n)$ is “one full lap” of the dial: you end up exactly back where you started, at 1. Raising m to the power $k\varphi(n)$ is “ k full laps” — still exactly back at 1, however many laps. Raising m to the power $ed = 1 + k\varphi(n)$ is therefore “ k full laps, plus one extra step” — and one extra step past “back where you started” is just $m^1 = m$. Choosing e and d with $ed \equiv 1 \pmod{\varphi(n)}$ is precisely choosing them so encrypting-then-decrypting is always some whole number of full laps plus exactly one step.

Example 5.2 (The exponent identity, verified with actual numbers). Take $n = 10$, so $\varphi(n) = 4$, and pick $e = 3$. We need d with $3d \equiv 1 \pmod{4}$; $d = 3$ works, since $3 \cdot 3 = 9 = 1 + 2 \cdot 4$ (so $k = 2$ here). Take $m = 7$ (coprime to 10). Encrypt-then-decrypt: $m^e = 7^3 = 343 \equiv 3 \pmod{10}$, then $(m^e)^d = 3^3 = 27 \equiv 7 \pmod{10}$ — back to the original $m = 7$, exactly as the identity above guarantees. Note $ed = 9 = 1 + 2 \cdot 4$: two full laps around a 4-position dial, plus one extra step.

This is *exactly* the decryption-undoes-encryption identity RSA needs, and the next subsection turns it into a key-generation recipe: pick e first, then solve $ed \equiv 1 \pmod{\varphi(n)}$ for d using the Extended Euclidean Algorithm (Theorem 2.7). There is one loose thread left, and it's worth being upfront about it rather than sweeping it under the rug: the derivation above assumed $\gcd(m, n) = 1$. A real message m might, in principle, share a factor with n , and then Euler's theorem as stated does not apply to it, since it is only a statement about the “well-behaved club” $(\mathbb{Z}/n\mathbb{Z})^*$. Section 5.3 closes this gap using the Chinese Remainder Theorem (Theorem 4.2) and proves the identity for *every* message.

5.2 Key Generation, Encryption and Decryption

Key generation sets up the numbers so that the identity of Section 5.1 applies; encryption and decryption are nothing more than *using* it.

Algorithm 1 RSA Key Generation

- 1: Choose two large distinct primes p, q of similar bit-length (typically 1536 bits each for a 3072-bit modulus)
 - 2: Compute $n = pq$
 - 3: Compute $\varphi(n) = (p-1)(q-1)$
 - 4: Choose e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
 - 5: Compute $d \equiv e^{-1} \pmod{\varphi(n)}$ via the Extended Euclidean Algorithm
 - 6: **Public key:** (n, e) **Private key:** (n, d) (or (p, q, d) for CRT speedup)
-

Definition 5.3 (RSA Encryption / Decryption). For a message $m \in \{0, 1, \dots, n-1\}$:

$$\text{Encryption: } c = m^e \bmod n, \quad \text{Decryption: } m = c^d \bmod n.$$

Remark 5.4 (Why $\gcd(e, \varphi(n)) = 1$ is required — and why this is a completely different condition from $\gcd(a, n) = 1$). It is easy to conflate two superficially similar-looking coprimality conditions in this lecture, so it is worth pulling them apart explicitly, side by side:

	messages / bases a	exponents e, d
live in	$\{0, \dots, n-1\}$	$\{0, \dots, \varphi(n)-1\}$
“nice” subset	a with $\gcd(a, n) = 1$	e with $\gcd(e, \varphi(n)) = 1$
why	lets Euler's theorem apply to a	lets e be <i>inverted</i> mod $\varphi(n)$

These are the same *shape* of condition (“coprime to the modulus you’re working in”), but applied to two genuinely different moduli, one level apart: bases are filtered against n , exponents are filtered against $\varphi(n)$. Do not read “ $1 < e < \varphi(n)$ ” as merely naming a range to pick a number from — the *range itself* is forced (exponents provably repeat with period $\varphi(n)$, exactly as bases repeat with period n ; see the paragraph below), but landing inside that range is not sufficient on its own: only the $\varphi(\varphi(n))$ values in that range that are coprime to $\varphi(n)$ are actually usable as an encryption exponent.

Why the range is $\{0, \dots, \varphi(n)-1\}$ at all. This is a separate fact from the coprimality filter, and it is worth seeing why exponents wrap around with period $\varphi(n)$, not n . For any a coprime to n and any integer x , Euler's theorem gives

$$a^{x+\varphi(n)} = a^x \cdot a^{\varphi(n)} \equiv a^x \cdot 1 = a^x \pmod{n},$$

so raising a to the power e and to the power $e + \varphi(n)$ produces the *identical* ciphertext, every time. Exponents, for the purposes of RSA, genuinely live in $\mathbb{Z}/\varphi(n)\mathbb{Z}$ — a clock with $\varphi(n)$ positions —

exactly parallel to how bases live in $\mathbb{Z}/n\mathbb{Z}$, a clock with n positions. Picking e larger than $\varphi(n)$ is not wrong, merely wasteful: it computes the exact same encryption function as $e \bmod \varphi(n)$, using more squarings to get there.

Why coprimality to $\varphi(n)$, specifically, is needed on top of that. We need e to be *invertible* modulo $\varphi(n)$, i.e. we need some d with $ed \equiv 1 \pmod{\varphi(n)}$. An element of $\mathbb{Z}/\varphi(n)\mathbb{Z}$ is invertible if and only if it is coprime to $\varphi(n)$ (Theorem 2.7, the same standard fact used throughout this lecture: $(\mathbb{Z}/k\mathbb{Z})^*$ consists exactly of residues coprime to k — here applied with $k = \varphi(n)$ rather than $k = n$). Without this, no valid d exists at all, and encryption under such an e could never be undone.

Example 5.5 (The filter, made fully concrete on $n = 10$). Take $n = 10$, so $\varphi(10) = 4$ and exponents reduce modulo 4: the only candidates are $e \in \{0, 1, 2, 3\}$. Check each one directly against $\gcd(e, 4)$:

e	0	1	2	3
$\gcd(e, 4)$	4	1	2	1

$e = 0$ is useless regardless ($m^0 = 1$ always, and it isn't invertible: $\gcd(0, 4) = 4 \neq 1$). $e = 1$ is invertible ($d = 1$) but trivial (“encryption” that does nothing). $e = 2$ has $\gcd(2, 4) = 2 \neq 1$ — **no d exists**: checking every value of $d \bmod 4$ directly, $2 \cdot 0 \equiv 0$, $2 \cdot 1 \equiv 2$, $2 \cdot 2 \equiv 0$, $2 \cdot 3 \equiv 2 \pmod{4}$, and 1 never appears, so $e = 2$ can never be decrypted and must be excluded. $e = 3$ has $\gcd(3, 4) = 1$, and indeed $3 \cdot 3 = 9 \equiv 1 \pmod{4}$, giving $d = 3$ — this is the only usable, nontrivial exponent for $n = 10$, and it is exactly the pair $e = d = 3$ of Example 5.2.

Notice the parallel exactly: just as the “nice” bases mod 10 are the coprime-to-10 subset $\{1, 3, 7, 9\} \subset \{0, \dots, 9\}$, the “nice” exponents mod $\varphi(10) = 4$ are the coprime-to-4 subset $\{1, 3\} \subset \{0, \dots, 3\}$ — the same filtering idea, one level up, applied to $\varphi(n)$ instead of n .

At realistic RSA sizes, $\varphi(n)$ has hundreds of digits, so this filter is never applied by listing candidates (as in the tiny example above) — it is applied by picking a candidate e (in practice, almost always the fixed value 65537) and directly testing $\gcd(e, \varphi(n)) = 1$ via the Euclidean algorithm, as in Section 2.2 (the table method of Section 2.4 extends it).

Example 5.6 (Toy example — never use these sizes in practice). Let $p = 61$, $q = 53$. Then $n = 3233$ and $\varphi(n) = 60 \cdot 52 = 3120$. Choose $e = 17$; the table method confirms $\gcd(17, 3120) = 1$ and produces $d = 2753$, since $17 \times 2753 = 46801 = 15 \times 3120 + 1$ (full table in Example 2.11). Encrypt $m = 65$: $c = 65^{17} \bmod 3233 = 2790$ (square-and-multiply trace in Example 5.13). Decrypt: $2790^{2753} \bmod 3233 = 65$. ✓

The private exponent d is computable only for someone who knows $\varphi(n)$, that is, someone who can factor n : this is where the security of RSA comes from.

5.3 Correctness of RSA

Recall the loose thread flagged at the end of Section 5.1: our derivation of $m^{ed} \equiv m \pmod{n}$ assumed $\gcd(m, n) = 1$. We now close that gap and prove $\text{Dec}(\text{Enc}(m)) = m$ for *every* $m \in \{0, \dots, n-1\}$ — not just those coprime to n . Most textbooks quietly assume $\gcd(m, n) = 1$ and invoke Euler's theorem directly; a rigorous treatment must handle the small-probability edge case, and this is exactly where CRT (Theorem 4.2) earns its keep: the strategy is to prove the congruence *separately* modulo p and modulo q (where a short case split handles the edge case cleanly) and then let CRT stitch the two into a single statement modulo $n = pq$.

Theorem 5.7 (RSA Correctness). *For all $m \in \{0, \dots, n-1\}$, $m^{ed} \equiv m \pmod{n}$.*

Proof. Since $ed \equiv 1 \pmod{\varphi(n)}$, write $ed = 1 + k\varphi(n) = 1 + k(p-1)(q-1)$ for some integer $k \geq 0$ (using $\varphi(pq) = (p-1)(q-1)$, Proposition 3.3).

By CRT (Theorem 4.2), it suffices to show

$$m^{ed} \equiv m \pmod{p} \quad \text{and} \quad m^{ed} \equiv m \pmod{q},$$

since p and q are coprime, these together are equivalent to $m^{ed} \equiv m \pmod{pq}$ (Lemma 2.4; this is the uniqueness half of CRT).

We show the first congruence; the second is symmetric. Consider two cases.

Case 1: $p \nmid m$. Then $\gcd(m, p) = 1$, so by Fermat's Little Theorem (Corollary 3.9) $m^{p-1} \equiv 1 \pmod{p}$. Hence

$$m^{ed} = m^{1+k(p-1)(q-1)} = m \cdot (m^{p-1})^{k(q-1)} \equiv m \cdot 1^{k(q-1)} \equiv m \pmod{p}.$$

Case 2: $p \mid m$. Then $m \equiv 0 \pmod{p}$, and since $ed \geq 1$, $m^{ed} \equiv 0 \equiv m \pmod{p}$ trivially.

Either way, $m^{ed} \equiv m \pmod{p}$. By the symmetric argument, $m^{ed} \equiv m \pmod{q}$. By CRT, $m^{ed} \equiv m \pmod{pq} = \pmod{n}$. $\square$

Remark 5.8 (Why this edge case matters pedagogically, not just practically). The event $p \mid m$ for a random $m \in \{0, \dots, n-1\}$ has probability $1/p$ — utterly negligible for cryptographic-size primes, so it is *never* a practical concern. But it matters conceptually: it shows correctness of RSA is really a CRT argument dressed up, and it is the reason the correctness proof cannot be “just Euler’s theorem” as often claimed. This distinction resurfaces when we discuss why RSA’s algebraic structure (a ring homomorphism argument, not merely a group-order argument) enables attacks like Håstad’s broadcast attack (Section 6.1).

Example 5.9 (The edge case, made concrete). With $n = 3233 = 61 \times 53$, the “bad” messages are the tiny fraction that are multiples of 61 or 53: e.g., $m = 61$. Here $p \mid m$, so Case 2 applies mod 61 ($m^{ed} \equiv 0 \equiv m \pmod{61}$ trivially), while Case 1 applies mod 53 (since $\gcd(61, 53) = 1$, Fermat’s little theorem does the work mod 53). CRT stitches the two together. You can verify numerically that $61^{17 \cdot 2753} \bmod 3233 = 61$ still holds — correctness survives even though Euler’s theorem alone (which needs $\gcd(m, n) = 1$) does not directly apply to $m = 61$, since $\gcd(61, 3233) = 61 \neq 1$.

Exercise 5.10. Prove the case $q \mid m$ directly (mirror the argument above), and explain why $p \mid m$ and $q \mid m$ cannot both hold when $m < n = pq$ and $m \neq 0$.

5.4 Fast Modular Exponentiation

Both encryption and decryption require computing $a^k \bmod n$ efficiently. Naive repeated multiplication is $O(k)$ multiplications; **square-and-multiply** reduces this to $O(\log k)$.

Example 5.11 (Right-to-left trace). Compute $5^{117} \bmod 19$. Write $117 = 1110101_2$ (bits, LSB first: 1, 0, 1, 0, 1, 1, 1). Trace of MODEXP, showing (bit consumed, *result*, *base* after squaring):

step	bit	result	base
1	1	5	6
2	0	5	17
3	1	9	4
4	0	9	16
5	1	11	9
6	1	4	5
7	1	1	6

Algorithm 2 Square-and-Multiply (Right-to-Left, two variables)

```

1: function MODEXP( $a, k, n$ )
2:    $result \leftarrow 1$ 
3:    $base \leftarrow a \bmod n$ 
4:   while  $k > 0$  do
5:     if  $k \bmod 2 = 1$  then
6:        $result \leftarrow (result \times base) \bmod n$ 
7:     end if
8:      $base \leftarrow (base \times base) \bmod n$ 
9:      $k \leftarrow \lfloor k/2 \rfloor$ 
10:  end while
11:  return  $result$ 
12: end function

```

Final result: 1. Direct check: $5^{117} \bmod 19 = 1$ (consistent: by Fermat's little theorem (Corollary 3.9) $5^{18} \equiv 1 \pmod{19}$, so only $117 \bmod 18 = 9$ matters, and $5^9 \equiv 1 \pmod{19}$ — try verifying this independently). Only **7 squarings and 5 multiplications** were needed instead of 116 successive multiplications.

Left-to-right square-and-multiply (one variable)

The right-to-left version above tracks two variables (*result* and *base*). The **left-to-right** version reads the bits of k starting from the *most significant* bit and needs only *one* variable, *result*. The idea: after reading some leading bits of k , *result* equals a raised to the number formed by those bits. Reading one more bit turns that number j into $2j$ (append a 0) or $2j + 1$ (append a 1), so we *square*, and if the new bit is 1 we also *multiply by a*.

Algorithm 3 Square-and-Multiply (Left-to-Right)

```

1: function MODEXPLTR( $a, k, n$ )
2:    $result \leftarrow 1$ 
3:   for each bit  $b$  of  $k$ , from the most significant to the least significant do
4:      $result \leftarrow (result \times result) \bmod n$  ▷ square
5:     if  $b = 1$  then
6:        $result \leftarrow (result \times a) \bmod n$  ▷ multiply
7:     end if
8:   end for
9:   return  $result$ 
10: end function

```

Recipe (by hand)

Write k in binary. Start with $result = 1$. For each bit, left to right: **(1)** square *result* and reduce mod n ; **(2)** if the bit is 1, multiply *result* by a and reduce mod n . After the last bit, $result = a^k \bmod n$.

Example 5.12 (Left-to-right trace: $5^{117} \bmod 19$). Here $117 = 1110101_2$, read from the left: 1, 1, 1,

0, 1, 0, 1. Start with $result = 1$.

step	bit	after squaring	after multiplying by 5
1	1	$1^2 = 1$	$1 \cdot 5 = 5$
2	1	$5^2 = 25 \equiv 6$	$6 \cdot 5 = 30 \equiv 11$
3	1	$11^2 = 121 \equiv 7$	$7 \cdot 5 = 35 \equiv 16$
4	0	$16^2 = 256 \equiv 9$	9 (skip)
5	1	$9^2 = 81 \equiv 5$	$5 \cdot 5 = 25 \equiv 6$
6	0	$6^2 = 36 \equiv 17$	17 (skip)
7	1	$17^2 = 289 \equiv 4$	$4 \cdot 5 = 20 \equiv 1$

Final result: $5^{117} \equiv 1 \pmod{19}$, the same answer as the right-to-left trace. (The multiply step is skipped when the bit is 0. Step 1 is trivial because $1^2 = 1$; many people start with $result = a$ and skip the first bit.)

Example 5.13 (Left-to-right trace: the RSA encryption $65^{17} \bmod 3233$). Here $17 = 10001_2$, so the bits are 1, 0, 0, 0, 1. Start with $result = 1$.

step	bit	after squaring	after multiplying by 65
1	1	1	65
2	0	$65^2 = 4225 \equiv 992$	992
3	0	$992^2 \equiv 1232$	1232
4	0	$1232^2 \equiv 1547$	1547
5	1	$1547^2 \equiv 789$	$789 \cdot 65 \equiv 2790$

So $c = 65^{17} \bmod 3233 = 2790$, matching the toy RSA example (Example 5.6). Only 4 squarings and 2 multiplications were needed, instead of 16 successive multiplications.

Remark 5.14 (Comparing the two versions). Both versions use about $\log_2 k$ squarings and one multiplication per 1-bit of k , and both give the same answer. Right-to-left keeps two variables ($result$, $base$) and reads bits from the least significant end. Left-to-right keeps one variable ($result$), reads bits from the most significant end, and always multiplies by the *same* fixed number a (useful when a is small, such as a message or a fixed base).

Remark 5.15 (Security-relevant note — preview of a later lesson). The naive implementations above have data-dependent branching (the `if` depends on the secret exponent's bits), which leaks timing information. This is the root cause of timing attacks on RSA, covered when we do implementation attacks.

5.5 CRT-Based Decryption Speedup

Since the private-key holder knows p and q , decryption can be done $\bmod p$ and $\bmod q$ separately (smaller exponents, smaller moduli) and recombined via CRT.

Why it works. By Theorem 5.7, $c^d \equiv m \pmod{n}$, hence also modulo p and modulo q . Fermat's little theorem (Corollary 3.9) lets us shrink the exponent: modulo p , powers of c repeat with period $p - 1$ when $p \nmid c$ (if $p \mid c$ both sides are 0, since $d_p \geq 1$ for odd p), so $c^d \equiv c^{d_p} \pmod{p}$ and therefore $m_p = m \bmod p$; likewise $m_q = m \bmod q$. The last two lines of the algorithm are the two-congruence formula of Theorem 4.6 with $a = m_q$ (first modulus q), $b = m_p$ (second modulus p), $u = q_{inv}$ and $t = h$: they rebuild the unique $m \in \{0, \dots, pq - 1\}$ with these two remainders (Theorem 4.2).

Algorithm 4 CRT-RSA Decryption

```

1: Precompute:  $d_p = d \bmod (p-1)$ ,  $d_q = d \bmod (q-1)$ ,  $q_{inv} = q^{-1} \bmod p$ 
2:  $m_p = c^{d_p} \bmod p$ 
3:  $m_q = c^{d_q} \bmod q$ 
4:  $h = q_{inv} \cdot (m_p - m_q) \bmod p$ 
5:  $m = m_q + h \cdot q$ 
6: return  $m$ 

```

Example 5.16 (CRT-RSA decryption, continuing the toy example). Recall $p = 61, q = 53, n = 3233, e = 17, d = 2753$, and $c = 2790$ (encryption of $m = 65$ in Example 5.6). Precompute:

$$d_p = d \bmod (p-1) = 2753 \bmod 60 = 53, \quad d_q = d \bmod (q-1) = 2753 \bmod 52 = 49, \\ q_{inv} = q^{-1} \bmod p = 38 \quad (\text{check: } 53 \cdot 38 = 2014 = 33 \cdot 61 + 1).$$

Then:

$$m_p = c^{d_p} \bmod p = 2790^{53} \bmod 61 = 4, \quad m_q = c^{d_q} \bmod q = 2790^{49} \bmod 53 = 12.$$

Recombine: $h = q_{inv}(m_p - m_q) \bmod p = 38 \cdot (4 - 12) \bmod 61 = 1$, and $m = m_q + h \cdot q = 12 + 1 \cdot 53 = 65$. ✓ Matches the direct decryption, but every intermediate exponentiation used a ~ 6 -bit exponent and a ~ 6 -bit modulus instead of the full-size d and n .

This gives roughly a $4\times$ speedup (two exponentiations with half-size exponents and half-size moduli, each individually $\sim 8\times$ cheaper, combined with cheap recombination). **Flag this in your memory:** this optimization is exactly what the Bellcore fault attack (covered in a later lesson) exploits — a single computational fault in either the m_p or m_q branch lets an attacker factor n from one faulty signature.

Example 5.17 (CRT-RSA decryption with tiny primes). We run Algorithm 4 with $p = 11, q = 7, n = pq = 77, e = 13, d = 37$ (indeed $ed = 481 = 8 \cdot 60 + 1$ and $\varphi(77) = 10 \cdot 6 = 60$) on the ciphertext $c = 52$, which is the encryption of $m = 24$ (check: $24^{13} \equiv 52 \pmod{77}$).

Line 1 (precompute).

$$d_p = 37 \bmod 10 = 7, \quad d_q = 37 \bmod 6 = 1, \quad q_{inv} = 7^{-1} \bmod 11 = 8,$$

since $7 \cdot 8 = 56 = 5 \cdot 11 + 1$.

Lines 2–3 (decrypt modulo each prime).

$$m_p = 52^7 \bmod 11 = 8^7 \bmod 11 = 2, \quad m_q = 52^1 \bmod 7 = 3.$$

Lines 4–5 (recombine). Here the gap $m_p - m_q = 2 - 3 = -1$ is negative, and the inverse q_{inv} is taken modulo the *other* modulus p (see the table of common slips at the end of Section 4.5):

$$h = 8 \cdot (2 - 3) = -8 \equiv 3 \pmod{11}, \quad m = m_q + h \cdot q = 3 + 3 \cdot 7 = 24. \quad \checkmark$$

This matches the direct decryption $52^{37} \equiv 24 \pmod{77}$, but each exponentiation used a tiny exponent (7 or 1) and a tiny modulus (11 or 7) instead of the full d and n .

Remark 5.18 (Two uses of CRT in this lecture — and why they are genuinely different jobs). CRT plays two completely different roles in RSA that students often conflate, precisely because both roles use the exact same theorem:

1. **Correctness** (Section 5.3): Euler's theorem alone only talks about m with $\gcd(m, n) = 1$. CRT lets us handle *every* message $m \in \{0, \dots, n-1\}$, including the astronomically rare ones sharing a factor with n , by checking correctness separately mod p and mod q (where a simpler, case-by-case argument works) and using CRT to stitch the two into one statement mod n .
2. **Efficiency** (Section 5.5): the private-key holder, who knows p and q , decrypts modulo p and modulo q separately and recombines the two partial results with the reconstruction formula of Theorem 4.6. This is a speed optimization and plays no part in correctness.

6 Four Attacks on RSA: Håstad, Common Modulus, Shared Primes, Fermat

Every attack below has the same shape: RSA is used with a small mistake in *how it is deployed or how its keys are generated*, and a tool from this lesson — the Chinese Remainder Theorem, the Extended Euclidean Algorithm, the plain gcd, or the difference of two squares — turns that mistake into a full break. The first two attacks never factor n at all. The last two do factor n , but only because a flawed key generator made n much easier to factor than a product of two independent random primes. None of them breaks correctly generated and correctly used RSA.

Attack	The mistake	Tool used	Fix
1. Håstad	Same message, small e , many recipients	CRT	Randomized padding (OAEP)
2. Common modulus	One n shared by several users	Extended Euclid	Never share n
3. Shared prime	Bad randomness reuses one prime p	gcd (Euclid)	Good entropy at key generation
4. Fermat	Primes p, q chosen too close together	Difference of squares	Pick p, q independently at random

How to read each Setup subsection. Every attack lists its conditions with labels: **P0** = system setup, **P1** = what the attacker can see, **P2** = mathematical or key conditions, **P3** = the mistake (behavior of a sender or a key generator), **P4** = the *exact* success condition (shown in bold), **P5** = a convenient *sufficient* condition that guarantees P4. The last column says whether the attacker can check the condition from public data before attacking.

6.1 Håstad's Broadcast Attack

Picture it

Think of $m^3 \bmod n$ as the reading on a clock with n hours. If m^3 is bigger than n , the hand goes around the dial and you only see where it ended up — that wraparound is what hides m . Now suppose the same m^3 is shown to you on *three different clocks*, of sizes n_1, n_2, n_3 . CRT lets you merge them into one giant clock of size $n_1 n_2 n_3$. If m^3 is smaller than that giant clock, the hand never wrapped, so the reading *is* m^3 , and a plain cube root gives m . Small e means too little wraparound to hide anything.

Key idea. CRT turns the k separate readings $c_i = m^e \bmod n_i$ into one reading $C \equiv m^e \pmod{N}$, where $N = n_1 n_2 \cdots n_k$. If in addition $m^e < N$, then m^e never wrapped around the big clock, so $C = m^e$ as ordinary integers, and an integer e -th root of C gives m . In short: CRT merges the clocks, the size condition removes the wraparound, and a plain root finishes the job.

When this arises. One message must go to several people: a vendor sends the same license key or firmware secret to many customers, or a group announcement is sent encrypted to each member. Every recipient has their own independent key pair, but many use the same tiny exponent (historically $e = 3$, chosen because it makes encryption fast on weak hardware). The sender encrypts the identical m separately under each recipient's public key.

Setup

Label	Condition	Type	Verifiable?
P0a	$k \geq 2$ distinct moduli	Setup	Yes
P0b	Same exponent e for all	Setup	Yes
P1	e known	Public key	Yes
P2	$\gcd(n_i, n_j) = 1$ for all $i \neq j$	Math / key	Yes
P3	$c_i = m^e \bmod n_i$ for one common m	Sender behavior	No (hypothesis)
P4	$m^e < N = \prod n_i$	Exact success condition	No (m unknown)
P5	$k \geq e$ and $m < \min(n_i)$	Sufficient guarantee for P4	Partly

Compact form. *Exact:*

$$e \text{ known, } \gcd(n_i, n_j) = 1, \ c_i = m^e \bmod n_i, \ m^e < \prod_{i=1}^k n_i$$

Guaranteed (implies the above):

$$k \geq e, \ \gcd(n_i, n_j) = 1, \ c_i = m^e \bmod n_i, \ m < \min_i n_i$$

One-line summary. The real condition is $m^e < \prod n_i$. The condition $m < \min(n_i)$ with $k \geq e$ is just a convenient *sufficient* version that guarantees the real one without knowing m . In the worked example below, $e = k = 3$.

Theorem 6.1. *Under P0–P4, an eavesdropper who sees $c_1, \dots, c_k$ recovers m with no private key. In particular this holds whenever $k \geq e$ and $m < \min_i n_i$ (P5).*

Why it works.

1. CRT combines the congruences $x \equiv c_i \pmod{n_i}$ into one unique answer C modulo $N = n_1 \cdots n_k$. This needs P2 and uses only public data.
2. The true value m^e satisfies all these congruences too, so by uniqueness $C \equiv m^e \pmod{N}$.
3. By P4, $m^e < N$. So C and m^e are both in $[0, N)$ and congruent mod N , hence *equal as ordinary integers*. (P5 implies P4: if $k \geq e$ and $m < \min_i n_i$, then $m^e < (\min_i n_i)^e \leq n_1 \cdots n_k = N$.)
4. Take an ordinary integer e -th root of C . Done.

Example 6.2 (Håstad, $e = k = 3$). Take $n_1 = 143 = 11 \cdot 13$, $n_2 = 323 = 17 \cdot 19$, $n_3 = 667 = 23 \cdot 29$, and secret message $m = 17$. The three ciphertexts are

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

Each number alone looks unrelated to 17. The attacker now solves

$$x \equiv 51 \pmod{143}, \quad x \equiv 68 \pmod{323}, \quad x \equiv 244 \pmod{667}$$

with CRT (Method C of Section 4.5, merging one pair at a time). Here $N = 143 \cdot 323 \cdot 667 = 30,808,063$.

Merge the first two. Take $a = 51$, $m = 143$, $b = 68$, $n = 323$. The inverse of 143 modulo 323 is $u = 192$ (Extended Euclid, or check: $143 \cdot 192 = 27456 = 85 \cdot 323 + 1$). Then

$$t \equiv (b - a)u = 17 \cdot 192 = 3264 \equiv 34 \pmod{323}, \quad x = 51 + 143 \cdot 34 = 4913,$$

so $x \equiv 4913 \pmod{143 \cdot 323}$, that is, modulo 46,189.

Merge with the third. Take $a = 4913$, $m = 46189$, $b = 244$, $n = 667$. Notice $4913 = 7 \cdot 667 + 244$, so $4913 \equiv 244 \pmod{667}$ already: the gap $b - a$ is $\equiv 0$, hence $t \equiv 0$ and nothing changes. So

$$C = 4913 \pmod{30,808,063}.$$

Since $17^3 = 4913$ is far less than $N \approx 3 \times 10^7$, there was no wraparound at all, and $\sqrt[3]{4913} = 17 = m$.

Takeaway and fix

Never send the same (or related) message to several recipients under a small e without randomized padding. With OAEP each ciphertext encrypts a *different* random-looking padded value, so the three equations no longer share one m^3 . (In general, with exponent e the attacker needs e recipients.)

6.2 Common Modulus Attack

Picture it

This is the classic water-jug puzzle. With a 17-litre jug and a 7-litre jug you can measure exactly 1 litre: fill the 7-litre jug five times and pour out the 17-litre jug twice ($5 \cdot 7 - 2 \cdot 17 = 1$). Here the “jugs” are the exponents. Ciphertext c_1 is m raised to the power 17, and c_2 is m raised to the power 7. Multiplying five copies of c_2 and “removing” two copies of c_1 leaves m raised to the power $5 \cdot 7 - 2 \cdot 17 = 1$, that is m itself.

Key idea. Two ciphertexts of the same m under the same n carry the exponents e_1 and e_2 “in the exponent”. Extended Euclid gives integers a, b with $ae_1 + be_2 = 1$, and then $c_1^a c_2^b \equiv m^{ae_1 + be_2} = m \pmod{n}$. The exponents combine like the jug measurements, and n is never factored.

When this arises. Generating a fresh modulus $n = pq$ for every user is slow, because finding large primes dominates the cost. A careless authority might generate n once and hand each user a different exponent pair (e_i, d_i) over that same n , thinking the different exponents keep users separate. This was a real anti-pattern in some early multi-user and embedded key-management designs.

Setup

Label	Condition	Type	Verifiable?
P0a	One modulus $n = pq$ shared by two users (the factorization stays secret)	Setup	Yes (compare public keys)
P0b	Two different exponents $e_1 \neq e_2$, one per user	Setup	Yes
P1	n, e_1, e_2, c_1, c_2 known	Public key / traffic	Yes
P2	$\gcd(e_1, e_2) = 1$	Exact success condition	Yes
P3	$c_1 = m^{e_1} \pmod{n}$ and $c_2 = m^{e_2} \pmod{n}$ for one common m	Sender behavior	No (hypothesis)
P4	$\gcd(c_1, n) = \gcd(c_2, n) = 1$ (so the inverse needed in step 3 exists)	Technical	Yes

Compact form.

$$n \text{ shared, } \gcd(e_1, e_2) = 1, \ c_1 = m^{e_1} \bmod n, \ c_2 = m^{e_2} \bmod n$$

One-line summary. The real condition is $\gcd(e_1, e_2) = 1$ on a shared modulus. Unlike Håstad, there is *no* size condition on m and no second “sufficient” version is needed. P4 is only a technicality: if it fails, $\gcd(c_i, n)$ is itself a prime factor of n , which is an even bigger break.

Theorem 6.3. *If $\gcd(e_1, e_2) = 1$, anyone who knows (n, e_1, e_2, c_1, c_2) recovers m with no private key.*

Why it works.

1. Extended Euclid on (e_1, e_2) gives integers a, b with $ae_1 + be_2 = 1$.
2. Then $c_1^a c_2^b \equiv m^{ae_1} m^{be_2} = m^{ae_1 + be_2} = m \pmod{n}$, by exponent laws alone.
3. One of a, b is negative. For a negative exponent, first compute the modular inverse of the corresponding ciphertext (again by Extended Euclid), then raise to the absolute value. (This is exactly P4: the inverse exists when $\gcd(c_i, n) = 1$.)

Notice that the attacker never factors n and never touches d_1 or d_2 .

Example 6.4 (Common modulus, $n = 3233$). The classic toy key $n = 3233 = 61 \cdot 53$, with $e_1 = 17$, $e_2 = 7$ and secret $m = 42$:

$$c_1 = 42^{17} \bmod 3233 = 2557, \quad c_2 = 42^7 \bmod 3233 = 240.$$

Step 1 — Extended Euclid on $(17, 7)$ (table method of Section 2.4):

i	q_i	r_i	s_i	t_i
0	—	17	1	0
1	—	7	0	1
2	2	3	1	−2
3	2	1	−2	5
4	3	0	7	−17

So $1 = (-2) \cdot 17 + 5 \cdot 7$, i.e. $a = -2$, $b = 5$. (Back-substitution gives the same: $17 = 2 \cdot 7 + 3$, $7 = 2 \cdot 3 + 1$, so $1 = 7 - 2 \cdot 3 = 7 - 2(17 - 2 \cdot 7) = 5 \cdot 7 - 2 \cdot 17$.)

Step 2 — handle the negative exponent. $c_1^{-1} \bmod 3233 = 3123$ (check: $2557 \cdot 3123 \equiv 1$).

Step 3 — combine.

$$m \equiv (c_1^{-1})^2 \cdot c_2^5 \equiv 2401 \cdot 1904 \equiv 42 \pmod{3233}.$$

An even worse consequence. Any *insider* who legitimately owns one pair (e_1, d_1) on the shared n can use the fact that $e_1 d_1 - 1$ is a multiple of $\varphi(n)$ to factor n (a standard randomized algorithm, not covered in this lesson). After that, every other user’s private key is compromised too, not just one message.

Takeaway and fix

Never share a modulus across users or purposes, even with different exponents. It is broken as soon as two exponents on it are coprime, which is the typical case.

6.3 Shared-Prime (Batch GCD) Attack

Picture it

Factoring a single 2048-bit number is like trying to un-bake a cake: hopeless. But if two cakes were baked with a common ingredient, you can find it just by comparing them — and Euclid’s algorithm does that comparison in a fraction of a millisecond. If two RSA moduli happen to share a prime, computing $\gcd(n_1, n_2)$ hands you that prime, and dividing gives the other prime of each. Both keys are broken, and neither owner did anything obviously wrong.

Key idea. If two moduli share a prime p , then $\gcd(n_1, n_2) = p$, and Euclid’s algorithm finds it in a handful of division steps without factoring anything. Once p is known, $q_1 = n_1/p$ and $q_2 = n_2/p$ follow, and the private keys of both owners can be computed.

When this arises. Devices such as routers, firewalls and other embedded systems often generate their RSA key the first time they boot. At that moment the operating system’s random number generator has had almost no entropy (no disk, no keyboard, no network jitter yet), so many devices produce *the same* first prime p . A little entropy trickles in (a clock tick, a packet) before the second prime is chosen, so their q ’s differ. The result is many public keys of the form $n_i = pq_i$ with a common p .

This is not hypothetical. In 2012 two independent teams (Lenstra et al., “*Ron was wrong, Whit is right*”; Heninger et al., “*Mining your Ps and Qs*”) collected millions of public RSA keys from the internet and factored thousands of them this way, with no ciphertext and no private key needed — just public moduli. To avoid comparing every pair, they used “batch GCD”, a product-tree algorithm that checks millions of moduli against each other in hours.

Setup

Label	Condition	Type	Verifiable?
P0a	At least two public moduli $n_1 \neq n_2$ collected (for example from many devices)	Setup	Yes
P0b	Each $n_i = p_i q_i$ is a valid RSA modulus with public exponent e_i	Setup	Yes
P1	Only the public data n_1, n_2 (and e_i , ciphertexts) known; the factorizations are unknown	Public key	Yes
P2	$n_1 \neq n_2$ (identical moduli give $\gcd = n_1$, which is useless)	Math / key	Yes
P3	$n_1 = pq_1$ and $n_2 = pq_2$ share exactly one prime p , with $q_1 \neq q_2$	Generator behavior (low entropy)	Yes (just compute the gcd)
P4	$1 < \gcd(n_1, n_2) < n_1$	Exact success condition	Yes
P5	A large collection of keys made by the same low-entropy generator (more keys, more pairs, more chance of a collision)	Makes P3 likely (heuristic)	Partly

Compact form.

$$n_1 \neq n_2, \gcd(n_1, n_2) > 1$$

One-line summary. The real condition is simply $\gcd(n_1, n_2) > 1$. Unlike the other attacks, it can be *checked in advance with public data*: given $n_1 \neq n_2$, P3 and P4 are equivalent, so an attacker just computes gcds (all pairs, or batch GCD for millions of keys) and needs no hypothesis about the senders.

Theorem 6.5. *If $n_1 = pq_1$ and $n_2 = pq_2$ share exactly one prime p , then $\gcd(n_1, n_2) = p$. Consequently $q_1 = n_1/p$ and $q_2 = n_2/p$, and the private keys of both moduli can be computed.*

Why it works.

1. p divides both n_1 and n_2 , so $p \mid \gcd(n_1, n_2)$.
2. Since q_1 and q_2 are distinct primes (and differ from p), $\gcd(n_1, n_2) = p \cdot \gcd(q_1, q_2) = p$.
3. With all primes known, compute $\varphi(n_1) = (p-1)(q_1-1)$ and $d_1 = e_1^{-1} \bmod \varphi(n_1)$ — exactly the recipe the legitimate key owner used. The same works for n_2 .

The gcd takes $O(\log^2 n)$ bit operations, whereas factoring a 2048-bit modulus directly is infeasible. That gap is the entire attack.

Example 6.6 (Shared prime, Alice and Bob). Alice's key is $n_1 = 3233 = 61 \cdot 53$ and Bob's key is $n_2 = 3599 = 61 \cdot 59$; both use $e = 17$. Eve sees only the two public moduli, so all she starts with is 3233 and 3599.

Step 1 — Euclid.

$$\begin{aligned} 3599 &= 1 \cdot 3233 + 366, \\ 3233 &= 8 \cdot 366 + 305, \\ 366 &= 1 \cdot 305 + 61, \\ 305 &= 5 \cdot 61 + 0, \end{aligned}$$

so $\gcd(3233, 3599) = 61$. Eve has found the shared prime $p = 61$ in four division steps.

Step 2 — factor both keys. $q_1 = 3233/61 = 53$ and $q_2 = 3599/61 = 59$.

Step 3 — rebuild Alice's private key. $\varphi(n_1) = 60 \cdot 52 = 3120$ and $d_1 = 17^{-1} \bmod 3120 = 2753$ (check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$; this is Example 2.11). Bob's key falls the same way: $\varphi(n_2) = 60 \cdot 58 = 3480$, $d_2 = 1433$.

Step 4 — read a message. Suppose someone sent Alice the ciphertext $c = 2183$. Eve computes $2183^{2753} \bmod 3233 = 1234$, the plaintext, without ever being given a key.

Takeaway and fix

Generate keys only after the system has gathered enough entropy, use the operating system's cryptographic random number generator, and draw p and q from fresh randomness. Avoid generating keys on the first boot of a device with no entropy source. Operators can also check their own newly generated keys against large public key collections for shared factors. Notice the lesson: RSA's security depends not only on the math but on the quality of the random numbers used to pick the primes.

6.4 Fermat's Factorization (Close Primes)

Picture it

Think of $n = pq$ as the area of a $p \times q$ rectangle of unit tiles. Any rectangle can be rearranged into a big square with a smaller square cut out of one corner. Let a be the *midpoint* of p and q and b the *distance from the midpoint to each*, so $p = a - b$ and $q = a + b$. Then the rectangle has the same area as a square of side a minus a square of side b :

$$pq = (a - b)(a + b) = a^2 - b^2.$$

If p and q are nearly equal, the rectangle is almost a square already, so a is only slightly above $\sqrt{n}$ and the cut-out square is tiny. The attacker simply tries $a = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ until $a^2 - n$ turns out to be a perfect square. That is a search from a known starting point, not a blind one.

Key idea. Write $n = pq = a^2 - b^2 = (a - b)(a + b)$ with $a = \frac{p+q}{2}$. If p and q are close, a is only slightly above $\sqrt{n}$, so testing $a = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ until $a^2 - n$ is a perfect square finds the factors after at most $\frac{(q-p)^2}{8\sqrt{n}} + 1$ tries.

When this arises. A key generator picks one random prime p and then, to save time or because of a flawed random source, picks q close to it — for instance the next prime after p , or p plus a small random offset. The idea itself dates to 1643, and for a long time it was regarded as a textbook curiosity. In 2022 Hanno Böck applied it to large collections of real public keys and found vulnerable ones: some Canon and Fujifilm (Fuji Xerox) printers generated keys with a cryptographic module that did not randomize its two primes enough (the Canon issue is tracked as CVE-2022-26351). It is a rare flaw, but it is real, and it needs nothing except the public modulus.

Setup

Label	Condition	Type	Verifiable?
P0a	$n = pq$ with $p < q$ distinct primes	Setup	Partly (form of n assumed)
P0b	n is odd (so p and q are odd)	Setup	Yes
P1	Only n (and e) is public	Public key	Yes
P2	$a = \frac{p+q}{2}$ and $b = \frac{q-p}{2}$ are integers with $a^2 - n = b^2$	Math / key	Yes (follows from P0b)
P3	p and q were generated close together by a flawed key generator	Generator behavior	No (hypothesis)
P4	Number of tries $\frac{p+q}{2} - \lceil \sqrt{n} \rceil + 1 \leq T$, the attacker's budget	Exact success condition	No (p, q unknown)
P5	$ q - p < \sqrt{8T} n^{1/4}$	Sufficient guarantee for P4	Partly

Compact form. *Exact:*

$$n = pq \text{ odd, } \frac{p+q}{2} - \lceil \sqrt{n} \rceil + 1 \leq T$$

Guaranteed (implies the above):

$$n = pq \text{ odd, } |q - p| < \sqrt{8T} n^{1/4}$$

One-line summary. Fermat's method always terminates, so the only question is *cost*: the real condition is that the number of tries stays within the budget T , and “ $q - p$ is small” is the convenient sufficient version (it follows from the bound in the theorem below, since tries $\leq \lfloor \frac{(q-p)^2}{8\sqrt{n}} \rfloor + 1 < T + 1$). With $T = 1$ it says $|q - p| < 2\sqrt{2}n^{1/4}$, which is about $2^{513.5}$ for a 2048-bit n . In the worked example $n = 5959$, $q - p = 42 < \sqrt{24}n^{1/4} \approx 43.0$, so $T = 3$ tries suffice.

The attack, step by step.

1. **Start** at $a = \lceil \sqrt{n} \rceil$. This skips nothing: every valid a satisfies $a^2 = n + b^2 \geq n$, so $a \geq \sqrt{n}$, and since a is an integer, $a \geq \lceil \sqrt{n} \rceil$. (As $p \neq q$ we have $b > 0$, so a is in fact strictly above $\sqrt{n}$.)
2. **Compute** $r = a^2 - n$.
3. **Test** whether r is a perfect square, that is, whether $b = \lfloor \sqrt{r} \rfloor$ satisfies $b^2 = r$.
4. **If yes:** $p = a - b$ and $q = a + b$. Check $pq = n$. Done.
5. **If no:** replace a by $a + 1$ and go back to step 2.

Theorem 6.7. Let $a = \frac{p+q}{2}$ and $b = \frac{q-p}{2}$ (integers, since p, q are odd). Then $a^2 - n = b^2$, and a is the smallest integer $\geq \sqrt{n}$ for which $a^2 - n$ is a perfect square. Testing $a = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ therefore finds a after at most

$$\frac{(q-p)^2}{8\sqrt{n}} + 1$$

tries, and then $p = a - b$, $q = a + b$.

Why it works.

1. Algebra: $a^2 - b^2 = (a - b)(a + b) = p \cdot q = n$, so $a^2 - n = b^2$ is a perfect square.
2. Conversely, if $a^2 - n = b^2$ for any integers $a > b \geq 0$, then $n = (a - b)(a + b)$ is a factorization. Since $n = pq$ has only one pair of nontrivial factors, the first hit gives exactly p and q .
3. How many tries? The search starts at $\lceil \sqrt{n} \rceil \geq \sqrt{n}$ and stops at $a = \frac{p+q}{2}$, so it takes at most $\frac{p+q}{2} - \sqrt{pq} + 1$ steps. Now

$$\frac{p+q}{2} - \sqrt{pq} = \frac{(\sqrt{q} - \sqrt{p})^2}{2} = \frac{(q-p)^2}{2(\sqrt{q} + \sqrt{p})^2} \leq \frac{(q-p)^2}{8\sqrt{n}},$$

because $(\sqrt{p} + \sqrt{q})^2 = p + q + 2\sqrt{n} \geq 4\sqrt{n}$ (as $p + q \geq 2\sqrt{pq}$).

So the work depends on the *gap* $q - p$, and grows with its square.

Example 6.8 (Fermat, $n = 5959$). Eve sees the public key $n = 5959$, $e = 17$. Here $\sqrt{5959} \approx 77.2$, so she starts at $a = 78$:

a	a^2	$a^2 - 5959$	perfect square?
78	6084	125	no ($11^2 = 121$, $12^2 = 144$)
79	6241	282	no ($16^2 = 256$, $17^2 = 289$)
80	6400	441	yes, $441 = 21^2$

So $a = 80$, $b = 21$, and $p = 80 - 21 = 59$, $q = 80 + 21 = 101$. Check: $59 \cdot 101 = 5959$. Three tries, and no factoring in the usual sense.

Finishing the break. $\varphi(n) = 58 \cdot 100 = 5800$ and $d = 17^{-1} \bmod 5800 = 3753$ (check: $17 \cdot 3753 = 63801 = 11 \cdot 5800 + 1$). If Alice's ciphertext is $c = 2029$, Eve computes $2029^{3753} \bmod 5959 = 1234$, the plaintext.

Contrast: primes that are far apart. Take $p = 101$, $q = 1009$, so $n = 101909$ and $\sqrt{n} \approx 319.2$. Now a must climb all the way to $\frac{101+1009}{2} = 555$ (with $b = 454$), which is 236 tries, against a bound of $\frac{908^2}{8\sqrt{n}} + 1 \approx 323$. The bigger the gap, the longer the climb.

How big a gap is safe? For a 2048-bit modulus, $8\sqrt{n} \approx 2^{1027}$. The bound above then says:

Gap $q - p$	Tries needed (at most)
below about 2^{512}	1
2^{512+t}	about $2^{2t}/8$ (each extra bit of gap quadruples the work)
about 2^{1023} (two independent random 1024-bit primes)	about 2^{1019} , hopeless

So Fermat only works when the two primes agree in roughly their top half of bits. Independent random primes never do.

Try it (about 10 lines of Python).

```
from math import isqrt

def fermat(n):
    a = isqrt(n)
    if a * a < n:          # a = ceil(sqrt(n))
        a += 1
    while True:
        b2 = a * a - n
        b = isqrt(b2)
        if b * b == b2:    # a^2 - n is a perfect square
            return a - b, a + b
        a += 1
```

`fermat(5959)` returns (59, 101). A good classroom demo: generate two 512-bit primes with `p = nextprime(r)` and `q = nextprime(p + 10**9)`. The 1024-bit modulus falls on the very first try.

Takeaway and fix

Choose p and q *independently* from a good random source, and reject the pair if they are too close. Standards such as FIPS 186-4 require $|p - q| > 2^{n_{\text{len}}/2 - 100}$. With independent random primes this is satisfied automatically, so the attack only ever hits a faulty generator, never a correct one. In particular, never build q from p , for example as `nextprime(p)` or $p + \text{small offset}$.